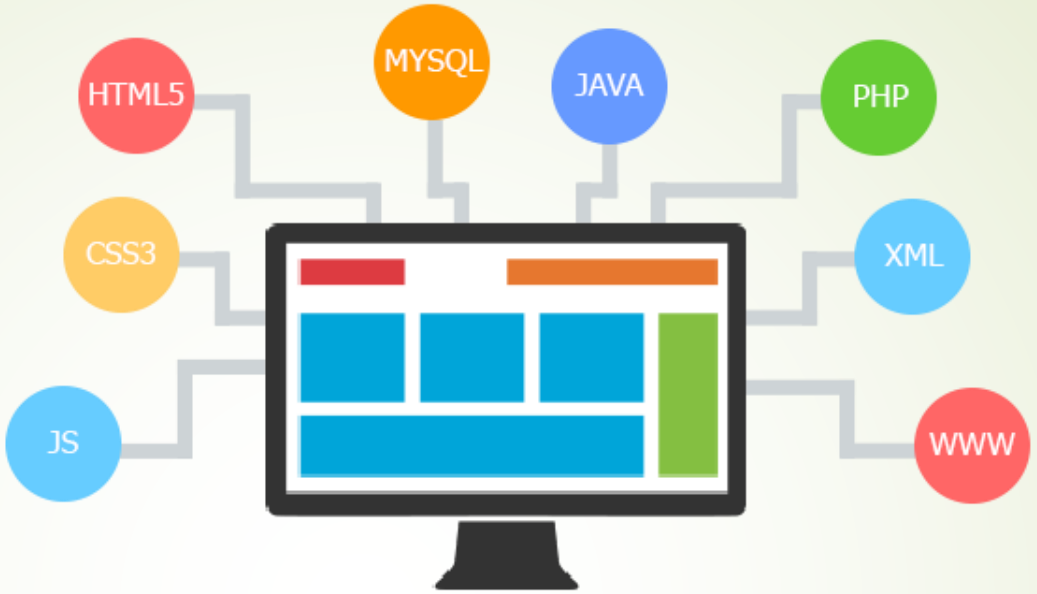
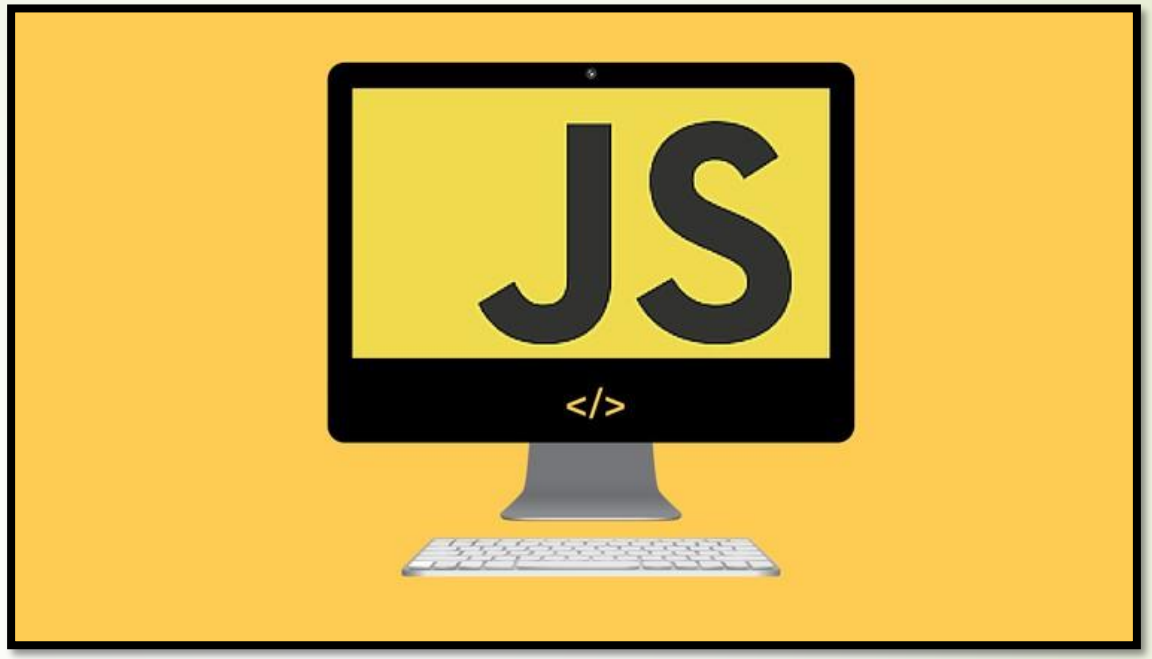


Desarrollo de Aplicaciones Web y Multiplataforma

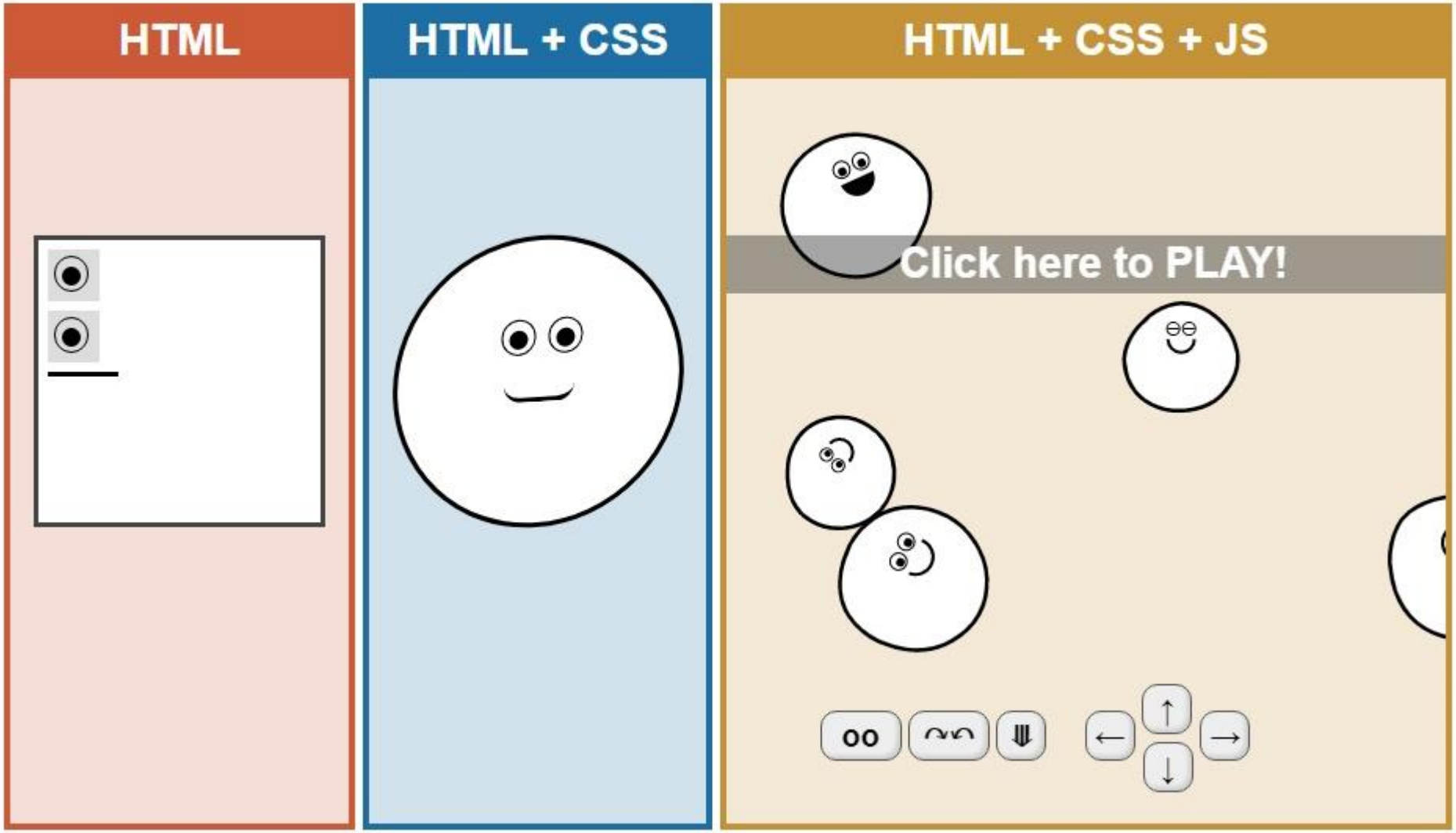
DOCENTE: Daniel López Lozano



Tema 1.
Introducción a JavaScript

Índice de contenidos

- Incluir JavaScript en una web
- Sintaxis básica. Variables y operadores
- String y Arrays
- if - elseif - else y switch
- while, do while y for
- Funciones


HTML
HTML + CSS
HTML + CSS + JS
Click here to PLAY!

Incluir JavaScript en una web

- El objetivo de JavaScript **es manipular el código HTML** y CSS, borrando, mostrando, modificando y añadiendo elementos y su presentación como **respuesta a una interacción** del usuario con la aplicación web.

- Existen 3 formas de incluir JavaScript en nuestro código HTML:
  - ✓ **Incrustado en etiquetas HTML.**
  - ✓ **La etiqueta <script>.**
  - ✓ **Ficheros externos js.**

Incrustado en etiquetas HTML

```
<body><p onclick="alert('Hola Mundo')">Texto de prueba.</p></body>
```

- Se usan para añadir comportamiento a los elementos HTML directamente.
- Mezcla el HTML con el JavaScript y complica el mantenimiento de la página.

Etiqueta <script>

```
<head>
    <script type="text/javascript">
        alert("Hola Mundo");
    </script>
</head>
```

- Se recomienda incluirlas **en la etiqueta head**, aunque nos encontraremos algunas excepciones necesarias.

Ficheros externos .js

```
Fichero hmtl:
<head>
    <script type="text/javascript" src="prueba.js"></script>
</head>
```

```
Fichero prueba.js:
alert("Hola mundo");
```

- Es la que utilizaremos nosotros en clase

El lenguaje **JavaScript.**

- Vamos a usar **JavaScript** porque es el lenguaje de script **más utilizado** en el lado del cliente, y está **soportado** mayoritariamente por todas las plataformas.
- **Brendan Eich** que trabajaba en el navegador Netscape desarrollo un lenguaje de programación llamado **Mocha**, que posteriormente se denomino **LiveScript**.
- Coincidiendo con la popularidad del **lenguaje Java** acabo denominándose **JavaScript**.
- ¿Entonces que es **ECMA Script**?

- Debido a que **Microsoft (JScript, VBScript)** y otros sacaron sus propias versiones, los autores **originales desarrollaron un estándar para la ECMA** que es el adoptado en la actualidad por todos los navegadores.
- De hecho **TypeScript** que ha sido desarrollado también por Microsoft es compatible con **JavaScript**, porque los dos se basan en **ECMA Script**.
- Otro lenguaje **ECMA Script** es **Babel** usado en **ReactJS**. No dejan de ser ampliaciones que convergen en JavaScript

Sintaxis básica, variables y operadores

- La sintaxis de JavaScript en la parte básica es **similar a la de C, C++, Java** y otros lenguajes relacionados.
- No se tiene en cuenta espacios en blanco y saltos de línea.
- Distingue las **mayúsculas y minúsculas**.
- Se pueden incluir comentarios con **// y /* */**
- Existen **palabras reservadas** del lenguaje.

- No es necesario **establecer el tipo de datos** en la declaración de las variables que se ajusta automáticamente.

- Los tipos de datos básicos existentes son:
  - ✓ **Numérico, incluyendo los números decimales.**
  - ✓ **Lógico o booleano, true o false.**
  - ✓ **Tipo cadena o string, que almacena texto.**
- El resto son de **tipo objeto**.

- El nombre de la variable solo puede contener letras, números y los símbolos **“_” y “$”**, además de no empezar por número. Para declarar variables se usa la palabra **let**.
- Existe otra forma de declarar variables con la palabra **var**, pero es desaconsejado su uso porque su alcance es confuso.
- Lo veremos más adelante con los bucles y funciones

```
let primera;
let puntuacion,record,jugador;
let correcta="ok";
let _correcta2=8;
let $correctisima=4.78;
let correcta_de_mas=true;
let 1incorrecta="blue";
let puntuacion=0,record=5000,jugador="Juan";
```

- Si probamos el código anterior vemos que primera no tiene valor y muestra undefined.

- Existen operadores aritméticos: +,-,*,/,%, ++,--,+=,-=, …

- De comparación: ==,!=,>,<,>=,<=, ===, !==

- Lógicos: !(NOT), &&(AND) y || (OR)

```
let cantidad,precio,total;
cantidad=100;
precio=5;
total=cantidad*precio;
```

Más ejemplos de operadores matemáticos

```
let num1=4,num2=5.4;
let suma=num1+num2;
let aumenta_cinco=num2+5;
let quita=num1-3;
let escrito="3";
let resultado=num1+escrito;
```

```
let base,exponente;
base=2;
exponente=5;
let potencia=base**exponente;
let resto=base%exponente;
```

- Otros operadores de asignación que incluyen operandos.

Operador	Utilización	Expresión equivalente
+=	op1 += op2	op1 = op1 + op2
-=	op1 -= op2	op1 = op1 - op2
*=	op1 *= op2	op1 = op1 * op2
/=	op1 /= op2	op1 = op1 / op2
%=	op1 %= op2	op1 = op1 % op2

¿Cuál es la diferencia entre == y ===?

- La principal diferencia es que === hace comparación de valores y de tipos, dando lugar a que si usamos == obtenemos lo siguiente:

```
0 == "0"      // true
132 == "132" // true
false == 0   // true
false == "0" // true
true == 1    // true
true == "1"  // true
"1,2" == [1,2] //true
```

- Tiene su lógica pero esconden un peligro

¿Cuál es la diferencia entre == y ===?

- Podría parecer que el motivo es perdida de eficiencia, pero realmente deriva a un problema de semántica muy grave:

```
null == undefined   // true
false == "" // true
false == [] // true
false == [0] // true
false == [[]] // true
0 == "" // true
```

- Conclusión usar siempre === y !==, pero no os riais de quienes no lo usan.

¿Cual es el valor de cada variable al ejecutar el código?

```
let A,B,C,D,E,F,G,H;
A=5;
B=3;
(A>B)
(A<B)
(A>=B)
(A<=B)
(A===B)
(A!==B)
```

```
4===4
4>3
x=5
x<=7
x<5
edad>=18
x===5
jornada=="media"
jornada!=="completa"
x%2===0
```

Operadores lógicos o booleanos

AND: sirve para UNIR

OR: sirve para ELEGIR

NOT: sirve para EXCLUIR

¿Cual es el valor de cada variable al ejecutar el código?

```
A=5;
B=3;
C=6;
D=1;
(C>A && D<B)
(C>A || D>B)
(!(D>B))
```

Tipo booleano en el mundo real

```
continuar=true;
!continuar

x>=1 && x<=10
x>45 && sexo=== "masculino"
x>45 || sexo=== "masculino"
letra=="A"||letra=="E"|| letra=="I"||letra=="O"|| letra=="U"
continuar && credito>0

encontrado=false;
fin=true;
encontrado || fin
```

Tipo booleano en el mundo real

```
alert(nombre===="Miguel");
alert(frase=="Eureka" );
alert(!aprobado);
alert(!fin);
alert(mes>=1 && mes<=12);
alert(ahorros>precio_coche || loteria==true );
alert(bachillerato==true  || prueba_acceso==true);
alert(isNaN(34));
alert(isNaN("dani"));
alert(!isNaN(10));
```

Utilización de valores constantes

```
//Si necesitamos guardar un valor y no va a cambiar
//es mas eficiente que una variable
const num1=4,num2=5.4;
const rojo="#FF0000";
const gravedad=9.8;
//darle significado a números
const pedido_recibido=0;
const pedido_procesado=1;
const pedido_enviado=2;
const arriba="w";
const abajo="s";
...
```

String y Array

Operaciones básicas de string

```
let texto="Un texto con 'comillas' dentro";
let otro_texto='Otro texto tambien con "comillas" dentro';
let numero_letras=text.length;

let prueba1="Hola "+ "Mundo";
let prueba2="Nota: "+ 10;
let prueba3="7"+5;
```

```
type(prueba1); //Devuelve el tipo
type(prueba2); //de una expresion o variable
type(prueba3);
```

Operaciones básicas de String

- Para acceder a un carácter en la posición pos, se debe usar **[pos]** o llamar al método **str.charAt(pos)**.
- El primer carácter comienza desde la posición cero:

```
let str = `Hola`;

// el primer carácter
str[0]; // H
str.charAt(0); // H
// el último carácter
str[str.length - 1]; // a
```

Template o plantilla

- Si usamos la comilla backticks o acento grave permite escribir en varia líneas.
- Son muy versátiles porque además pueden contener marcadores, identificados por el signo de dólar y envueltos en llaves **(${expresión}).**
- Las expresiones contenidas en los marcadores, junto con el texto entre ellas, son enviados como argumentos a una función

Template o plantilla

```
let a = 5;
let b = 10;
'Quince es ' + (a + b) + ' y no ' + (2 * a + b) + '.';
//Con templeates mas sencillo
`Quince es ${a + b} y no ${2 * a + b}.`;

//Si queremos salto de linea
'Quince es ' + (a + b) + ' y\nno ' + (2 * a + b) + '.';

`Quince es ${a + b} y
no ${2 * a + b}.`;
```

console.log

- Para escribir algo en la consola, existe la función console.log
- Los usuarios regulares no ven este output, ya que está en la consola. Para verlo, debemos abrir la consola de desarrolladores y presionar la tecla Esc y en otro tab: se abrirá la consola debajo.

```
let x = 1;
console.log("x:", x);
```

console.log y otras variantes

```
let x = 1;
let y = 2;
let z = 3;
console.log("x:", x, "y:", y, "z:", z);
console.log('Quince es ' + (a + b) + ' y\nno ' + (2 * a + b) + '.');
console.log(`Quince es ${a + b} y
no ${2 * a + b}.`);
```

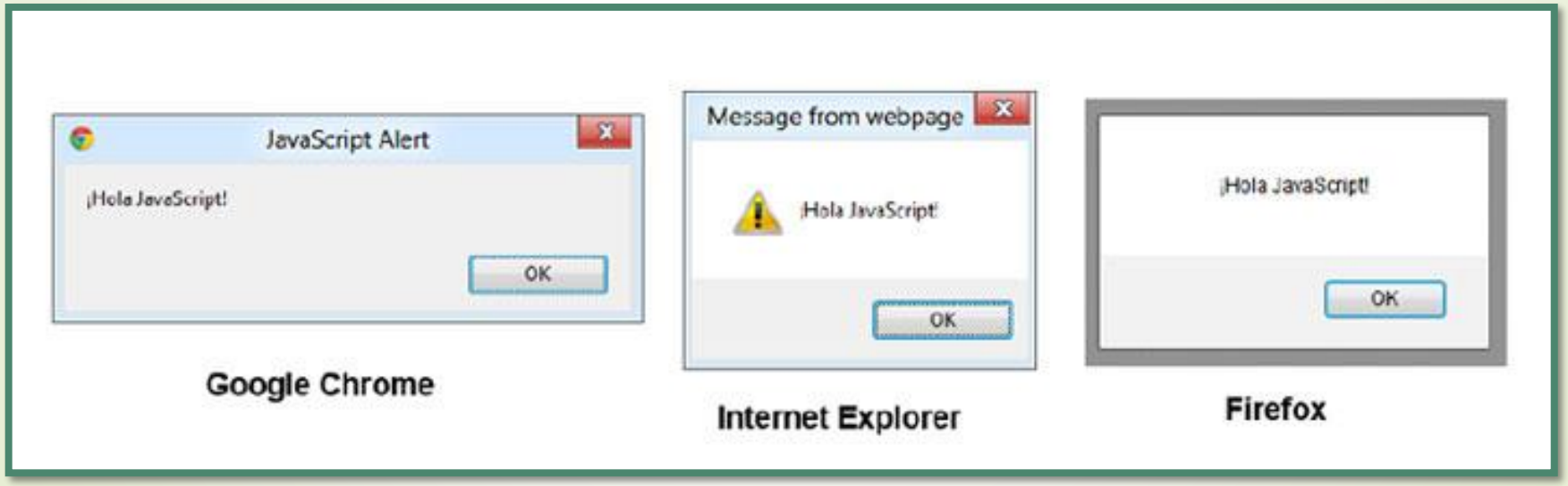
```
console.log('Consola de Registro');
console.info('Consola de Informacion');
console.debug('Console de Depuracion');
console.warn('Consola de Aviso');
console.error('Consola de Error');
```

```
Consola de Registro
Consola de Informacion
Consola de Aviso
Consola de Error
```

Falta un mensaje ¿por qué?

alert

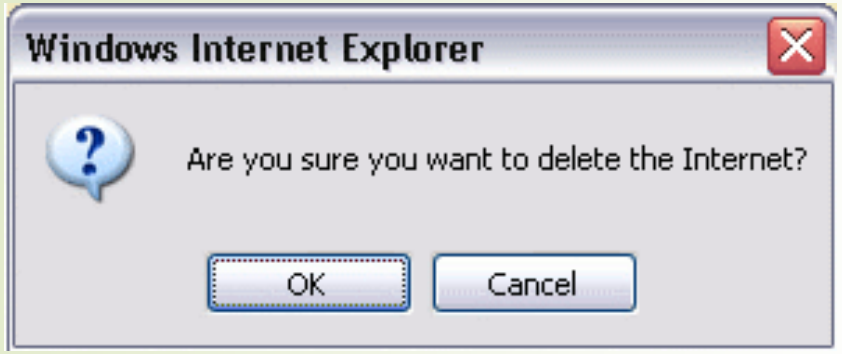
- **Muestra una ventana** con el mensaje que le hayamos indicado y es una forma básica de comunicación.

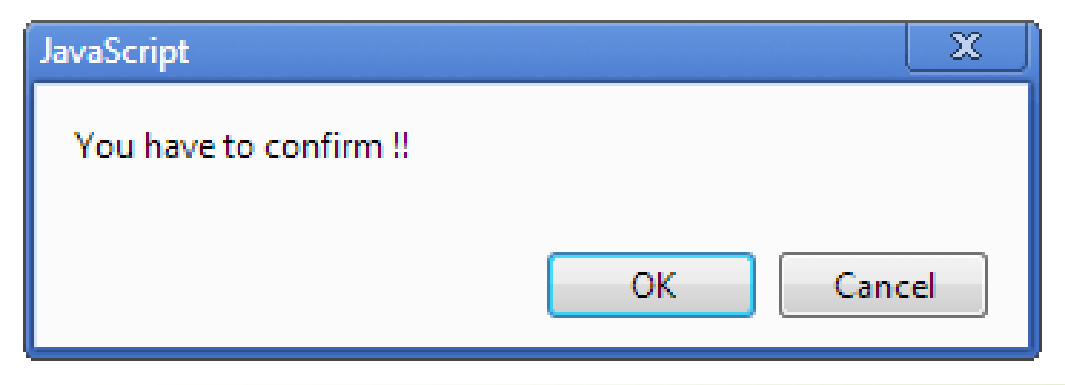


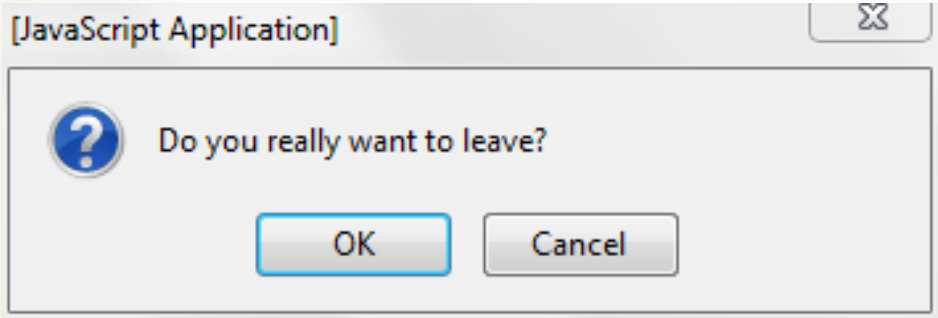
- Ya hemos visto alert para mostrar información, también existen mas métodos para pedir y mostrar información.

- La instrucción **prompt** además de mostrar un mensaje permite recoger información del usuario

- La orden **confirm** es parecida a la anterior pero ofreciendo la respuesta si o no.



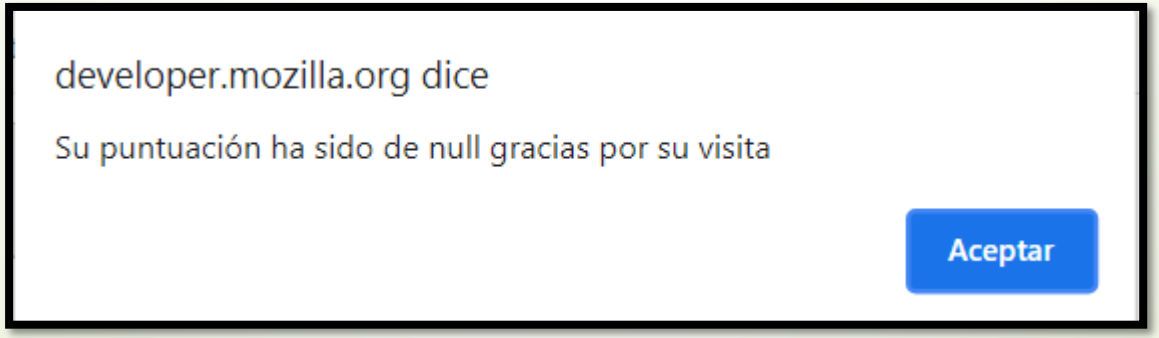




Entrada y salida de datos

```
let respuesta=confirm("Esta satisfecho con la aplicacion");
alert(respuesta);
let puntuacion=prompt("Puntue la aplicacion del 1 al 10");
alert("Su puntuacion ha sido de "+ puntuacion+" gracias por su visita");
```

- Si en el prompt no escribimos nada se devuelve la cadena vacía o si le damos a cancelar se devuelve null.



- **document** es una variable de tipo objeto que tiene un método llamado **write** que permite escribir código html en el **body de la página**.

- Por lo tanto cualquier etiqueta html que incluyamos se **interpretará** en el navegador. La idea es mezclar información de la variables con el código HTML.

```
document.write("Hola mundo");
document.write("<h1>Hola mundo</h1>");
let nota=7;
document.write("Tu nota es un <strong>"+nota+"</strong>");
```

Entrada y salida con JavaScript

```
document.write("Bienvenido a mi pagina web");
document.write("<h1>Bienvenido a mi pagina web</h1>");
let mensaje=prompt("Escriba una frase:");
document.write("<h1>"+mensaje+"</h1>");
let nivel=prompt("¿Que nivel de encabezado desea?(1-6)");
document.write("<h"+nivel+">Bienvenido a mi pagina web</h"+nivel+">");
```

El código se entiende, pero la concatenación es un poco tediosa

Los templates mejoran el código

```
let mensaje=prompt("Escriba una frase:");
document.write(`<h1>${mensaje}</h1>`);
let nivel=prompt("¿Que nivel de encabezado desea?(1-6)");
document.write(`<h${nivel}>Bienvenido a mi pagina web</h${nivel}>`);
```

Esta línea no daría el resultado esperado

Sugerencia: **probar parseInt**

```
document.write(`${nivel+1}`);
```

Depuración básica usando

```
alert(document);
alert(Math)
```

Esta página dice

[object HTMLDocument]

Aceptar

Esta página dice

[object Math]

Aceptar

```
console.log(document)
console.log(Math)
//pasa lo mismo con array
//o cualquier objeto
```

```
▼#document
    <!DOCTYPE html>
    <html>
    ▶<head>…</head>
    ▶<body>…</body>
    </html>

▼Math {abs: ƒ, acos: ƒ, acosh:
    E: 2.718281828459045
    LN2: 0.6931471805599453
    LN10: 2.302585092994046
    LOG2E: 1.4426950408889634
    LOG10E: 0.4342944819032518
    PI: 3.141592653589793
    SQRT1_2: 0.7071067811865476
    SQRT2: 1.4142135623730951
  ▶abs: ƒ abs()
  ▶acos: ƒ acos()
  ▶acosh: ƒ acosh()
  ▶asin: ƒ asin()
  ▶asinh: ƒ asinh()
  ▶atan: ƒ atan()
  ▶atan2: ƒ atan2()
```

- JavaScript cuenta con el tipo **Array** como un tipo básico.
- Un Array es una **colección de variables**, que se engloba en una misma variable.
- Los Arrays **son dinámicos** por lo que pueden cambiar de tamaño y pueden incluir cualquier tipo de datos.
- Los Arrays empiezan a numerarse en 0 y tiene la propiedad length como el tipo string.
- De hecho un **string** es un caso particular de un **Array**.

- De manera práctica son conjuntos de datos almacenados bajo el mismo nombre.

- Una representación gráfica de un array seria la siguiente

Posición	0	1	2	3
Valor	25	36	21	58

- Para crear el array representado en la imagen anterior es necesario este código.

```
let numeros=[];
    numero[0]=25;
    numero[1]=36;
    numero[2]=21;
    numero[3]=58;
```

- Un array en JavaScript no tiene un tamaño predefinido, con lo que puede tener tantos valores como sea necesario.
- Al final del tema 1 y en el tema 2 veremos toda versatilidad de los arrays en Javascript.

Operaciones básicas de un Array

```
let nombre_array=[];
let dias_lectivos=["Lunes","Martes","Miercoles","Jueve","Viernes"];
alert(dias[0]);
alert(dias.length);
let variado=["Dia",45,32.78,true];
let elMartes=dias_lectivos[1];

//un string tambien es un array
let nombre="Pepe"
alert(nombre[2]);
```

Crear HTML desde un array

Posición	0	1	2	3
Valor	León	Seat	5	V6

```
let coche=[];
    coche[0]="Leon";
    coche[1]="Seat";
    coche[2]=5;
    coche[3]="V6";

document.write("<ul>"+
                "<li>"+coche[0]+"</li>"+
                "<li>"+coche[1]+"</li>"+
                "<li>"+coche[2]+"</li>"+
                "<li>"+coche[3]+"</li>"+
               "</ul>");
```

Los templates funcionan muy bien con los arrays

```
let coche=["Leon","Seat",5,"V6"];
document.write(`<ul>
                <li>${coche[0]}</li>
                <li>${coche[1]}</li>
                <li>${coche[2]}</li>
                <li>${coche[3]}</li>
               </ul>
              `);
```

Dando lugar a crear un código mucho más claro y sencillo.

if - elseif - else y switch

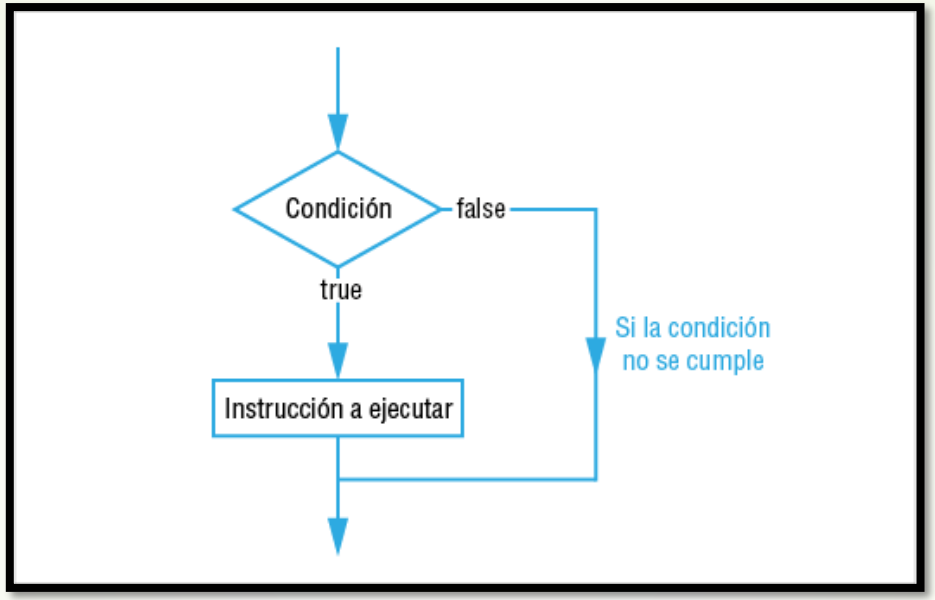
- Las estructuras condicionales realizan unas acciones u otras dependiendo del **estado de las variables**.

- Vamos a ver las estructuras **if-elseif-else y switch-case**

- La principal diferencia es que la estructura **if-else** comprueba si una **condición de cualquier tipo** es true o false.

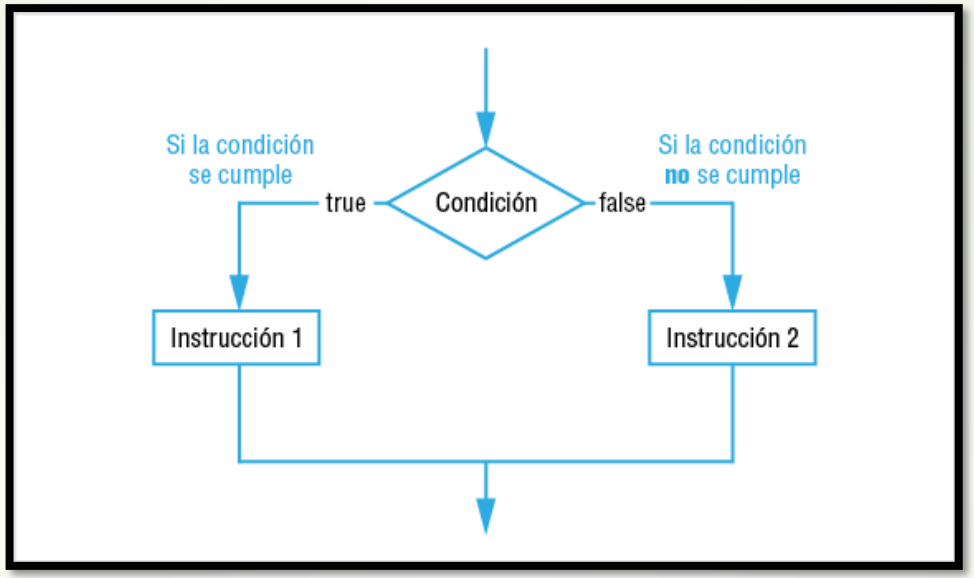
- Mientras que la estructura **switch** solo permite definir **condiciones de igualdad** con varios valores concretos.

Esquemas if-else



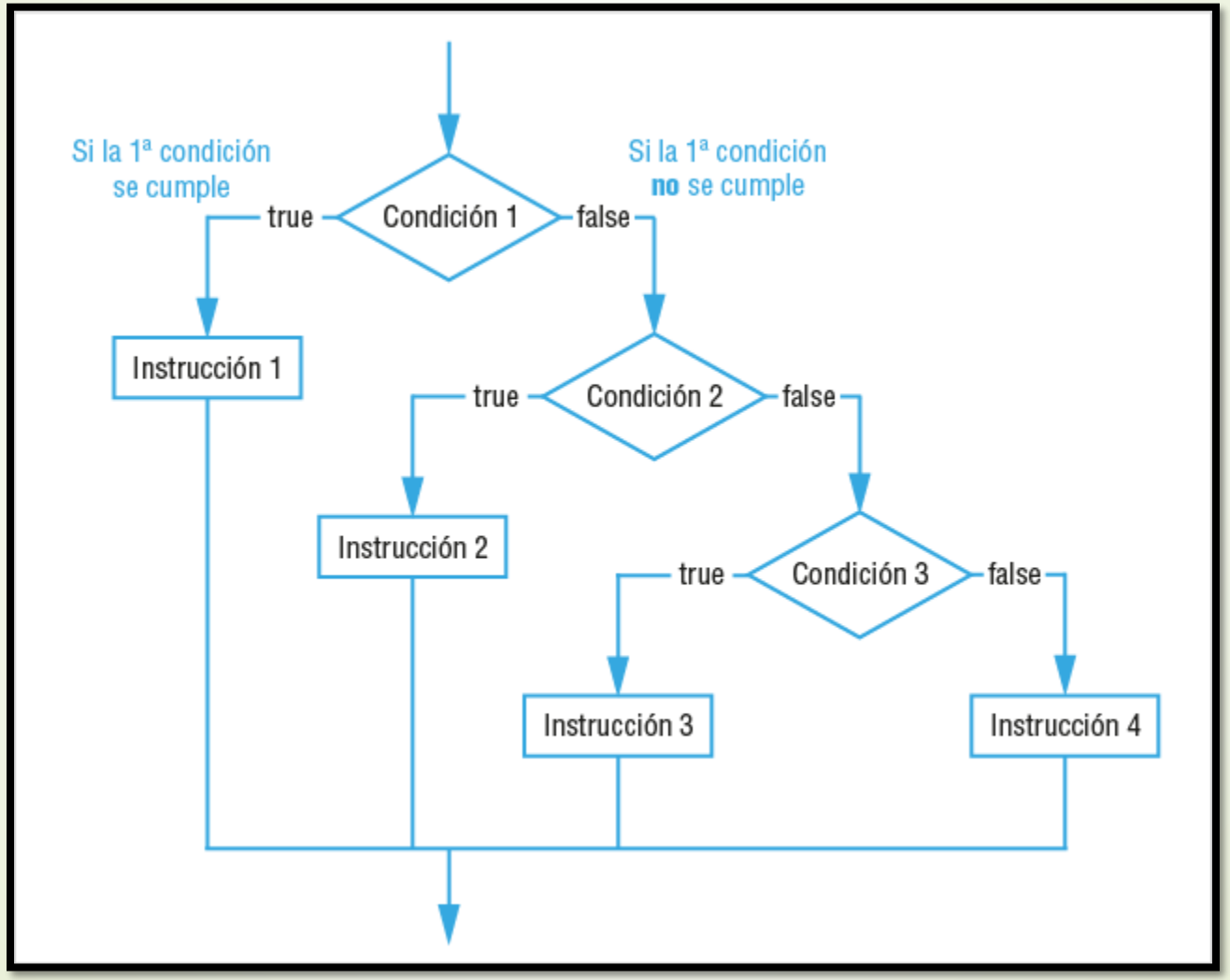

```
let miEdad=....
if(miEdad>30){
  console.log("Ya eres una persona adulta");
}
```

Esquemas if-else




```
let semaforo=...
if(semaforo=="verde")
{
    console.log("Puede pasar");
}else{
    console.log("Detengase");
}
```

Esquemas if-else



- Una forma de hacer una comprobación más completa seria la siguiente.

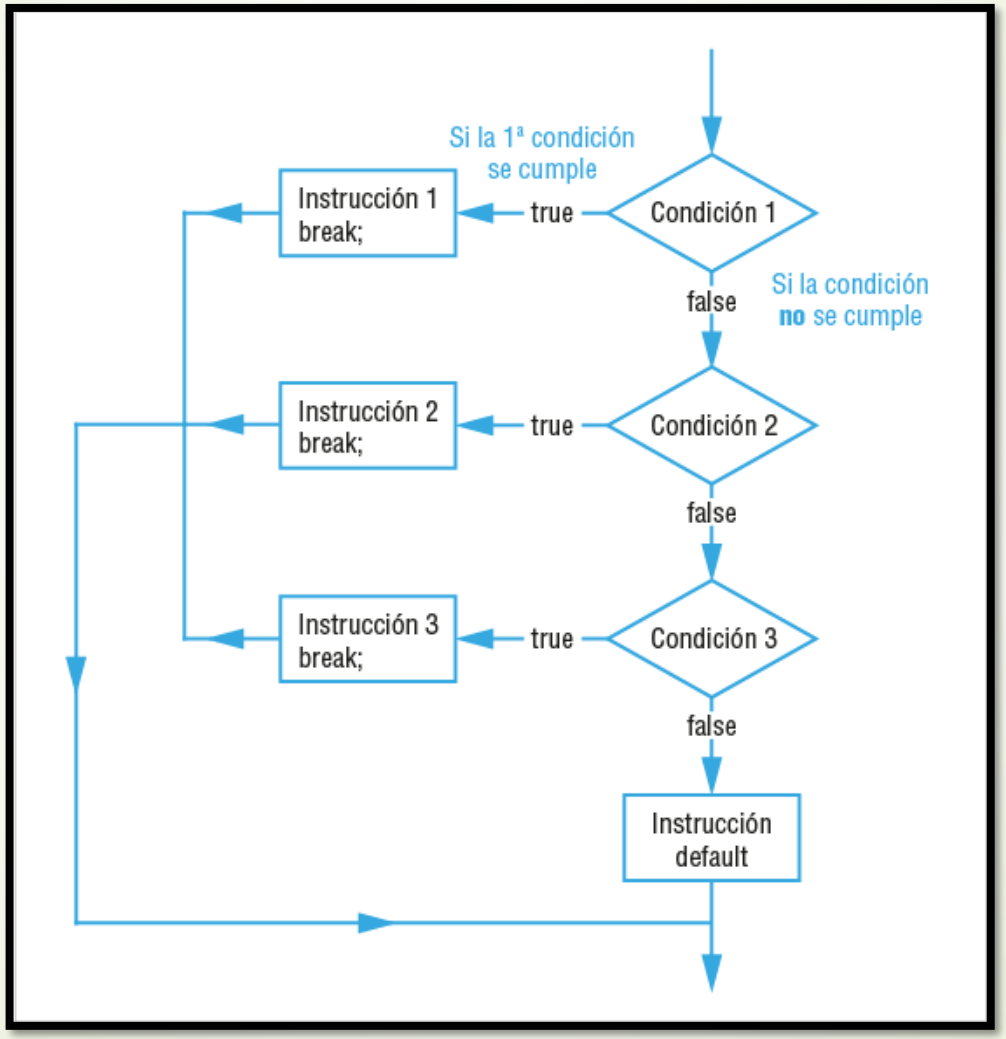
```
if(nota>=5)
{
  if(nota<7)
  {
    console.log("Aprobado</h2>");
  }else{
    console.log("Notable</h1>");
  }
}else{
  console.log("Suspenso</h3>");
}
```

- De manera equivalente se puede usar la forma if-elseif

```
if(nota>=7)
{
    console.log("Notable</h1>");
}else if(nota>=5){
    console.log("Aprobado</h2>");
}else{
    console.log("Suspenso</h3>");
}
```

```
let resolucion = …
if(resolucion<400)
{
   console.log("Resolucion movil");
}else if(resolucion<800){
   console.log("Resolucion tabler");
}else if(resolucion<1280){
   console.log("Resolucion tablet");
}else{
  console.log("Resolucion monitor");
}
```

Esquema switch



Esquemas switch

```
let dia=prompt("Introduce el dia de la semana");
switch(dia) {
    case 1:
        alert("Lunes");
    break;
    case 2:
        alert("Martes");
    break;
    ...
    default:
        alert("No es un dia de la semana");
}
```

Comparación if y switch

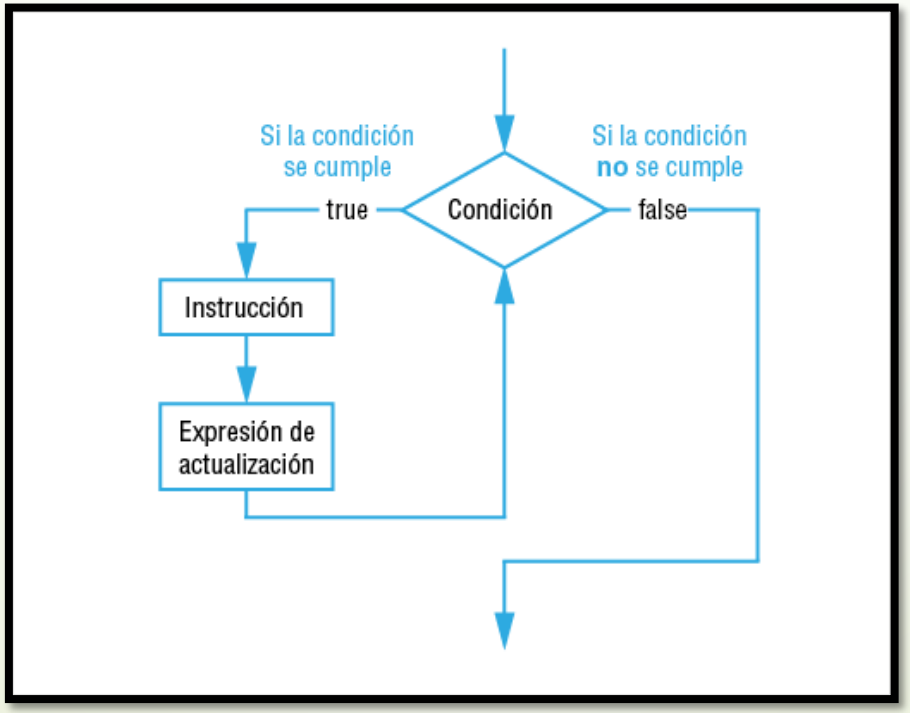
```
let materia=...
if(materia=="matematicas")
{
   console.log("Va de numeros");
}else if(materia=="lengua"){
   console.log("Va de letras");
}else if(materia=="ingles"){
   console.log("Va de hablar");
}else if(materia=="ciencias"){
   console.log("Va de investigar");
else{
   console.log("No sé de que va");
}
```

```
let materia=...
switch(materia)
{
   case "matematicas":
     console.log("Va de numeros");
     break;
   case "lengua":
     console.log("Va de letras");
     break;
   case "ingles":
     console.log("Va de hablar");
     break;
   case "ciencias":
     console.log("Va de investigar");
     break;
   default:
     console.log("No sé de que va");
}
```

while, do while y for

- Con el bucle for, podemos repetir una tarea un número fijo de veces.

- El bucle while permite crear bucles que se ejecutan cero o más veces de manera indefinida/ilimitada.



Ejemplos while

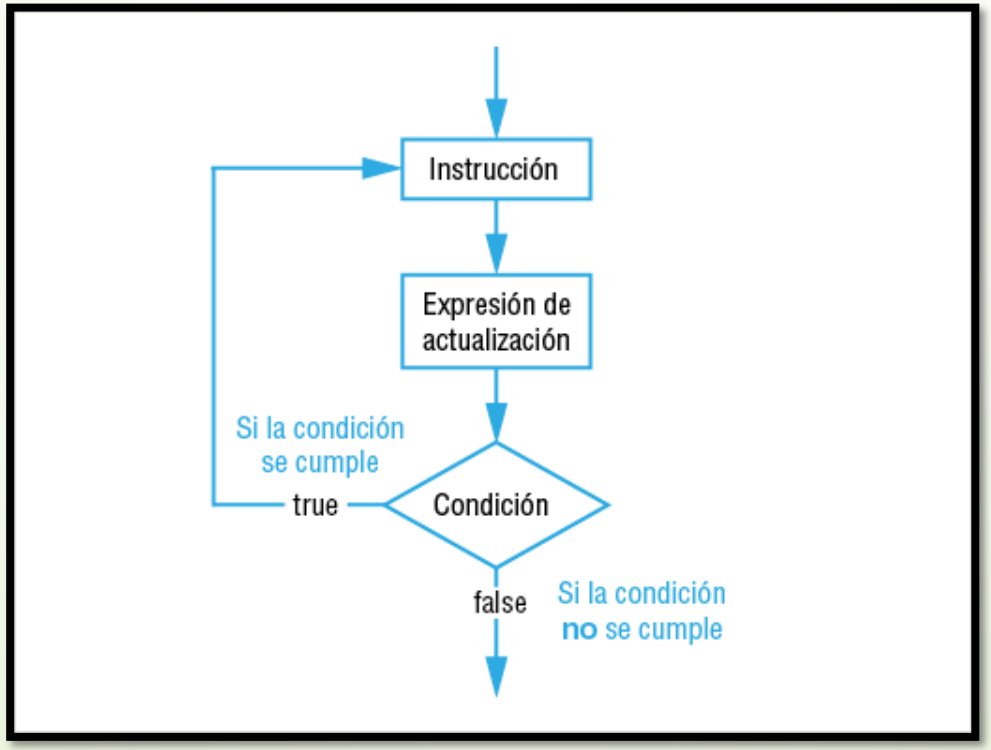
```
let i=0;
while(i<10)
{
  console.log("<br>El numero es " + i);
  i++;
}
```

```
let edad=prompt("Introduce tu edad");
while(edad<0 || isNaN(edad))
{
   edad=prompt("Introduce tu edad");
}
```

```
let secreta="eureka";
let intento="";
while(intento!==secreta)
{
   intento=prompt("¿Que palabra es?");
}
```

- El bucle do-while es una variante del bucle while que se ejecuta siempre al menos una vez.

- Permite escribir de manera más clara en ciertas situaciones



Ejemplos do-while

```
let i=1;
let respuesta;
do
{
   respuesta=confirm("¿Desea salir? Intento"+ i);
   i++;
}while (respuesta!==true);
```

```
let color;
do
{
  color=prompt("indica un color distinto de blanco");
}while(color==="white"|| color===="#FFF" || color===="#FFFFFF" );
```

- El bucle for permite ejecutar un bloque de código un número fijo y conocido de veces.

- Ese numero fijo de veces se establece mediante un condición de <= ó =>.

```
for (expresión inicial; condición; incremento)
{
    // Instrucciones a ejecutar dentro del bucle.
}
```

Bucle for

```
for(let veces=1;veces<=10;veces++){
  console.log(`El nombre numero ${veces} es ${nombre}`);
}
```

```
for(let i=1;i<=6;i++){
  document.write(`<h${i}>Cabecera ${i}</${i}h>`);
}
```

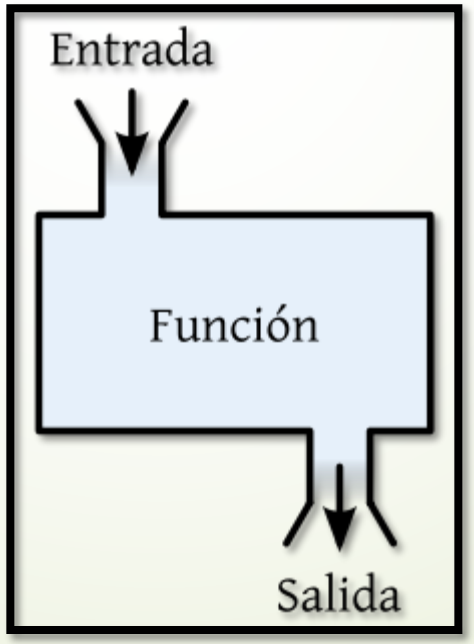
Array con for y sus variantes

```
let lenguajes=["HTML","CSS","JavaScript"];
//Solo para arrays
for(let i=0;i<lenguajes.length;i++)
{
   document.write(`<input type='button' value='${lenguajes[i]}'>`);
}

//funciona en array y objetos, no en colecciones
for(let posicion in lenguajes)
{
   document.write(`<input type='button' value='${lenguajes[posicion]}'>`);
}

//funciona en arrays y colecciones, no en objetos
for(let elemento of lenguajes)
{
   document.write(`<input type='button' value='${elemento}'>`);
}
```

Modularización de código. Las funciones

- Una función es una herramienta que nos permite definir una serie de instrucciones como si fuera solo una evitando así duplicar código.

- Utilizar funciones permite modularizar el código y hacer que sea más entendible, además de ser necesario para la gestión de eventos.



Funciones en JavaScript

```
function nombreFuncion(parametro1,parametro2,...)
{
    ...
}
```

```
function SumarIVA(cantidad,porcentaje)
{
    let  total;

    total=cantidad+cantidad*porcentaje/100;
    alert(total);
}
SumarIVA(400,18);
```

```
function ElMayor(num1,num2)
{
    if(num1>num2)
    {
        alert(num1+" es el mayor");
    }else{
        alert(num2+" es el mayor");
    }
}
ElMayor(4,7);
```

Diseño modular de funciones
No incluir entrada salida en funciones

```
function SumarIVA(cantidad,porcentaje)
{
   let  total;

   total=cantidad+cantidad*porcentaje/100;
   return total;
}

let resultado=SumarIVA(400,18);
//Puedo sacarla por pantalla por HTML o lo que quiera
```

Diseño modular de funciones

```
function ElMayor(num1,num2)
{
    let mayor;
    if(num1>num2)
    {
        mayor=num1;
    }else{
        mayor=num2;
    }
    return mayor;
}

let ganador=ElMayor(4,7);
//Puedo sacarla por pantalla por HTML o lo que quiera
```

Creando HTML con funciones modulares

```
function ejemploModular(datos,tipo){
    let res;
    if(tipo=="button"){
        res="";
        for(let valor of datos){
          res+=`<input type="button" value="${valor}"><br>`;
        }
    }else if(tipo=="select"){
        res="<select>";
        for(let valor of datos){
          res+=`<option value="${valor}">${valor}</option>`;
        }
        res+="</select>";
    }else{
        res="<h2>Tipo incorrecto</h2>";
    }

    return res;
}

let paises=["España","Francia","Portugal","Italia"];
let tipo= ...
document.write(ejemploModular(paises,tipo));
```

Parámetros opcionales

```
//Maneras tradicionales
function mostrarMensaje(persona,texto) {
  if(texto === undefined){
    texto="sin texto dado";
  }
  console.log(persona + ": " + texto);
}

function mostrarMensaje(persona,texto) {
  //texto undefined se considera false
  texto=texto || "sin texto dado";
  console.log(persona + ": " + texto);
}

//Con ECMA6

function mostrarMensaje(persona, texto = "sin texto dado") {
  console.log(persona + ": " + texto);
}

mostrarMensaje("Daniel"); // Daniel: sin texto dado
```

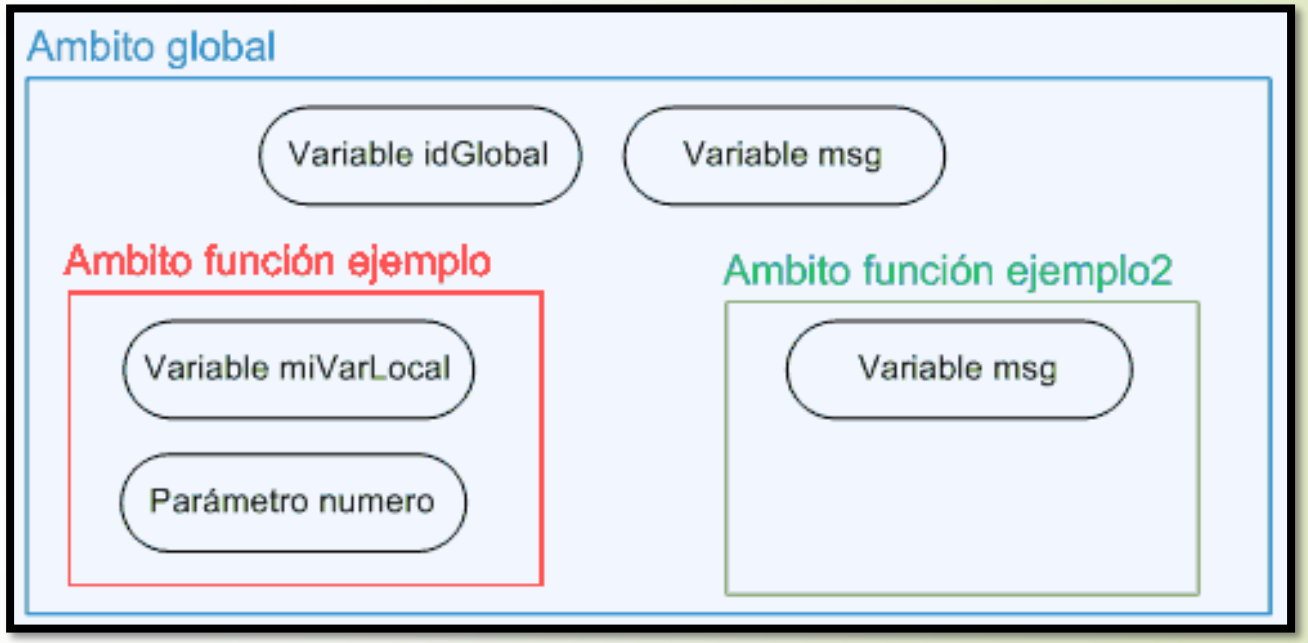
- El ámbito de una variable, es la zona del programa donde existe la variable y se puede operar con ella.

- **Variables locales:** Son variables como el contador de un bucle for o declaradas dentro de una función. **No** se pueden acceder desde **fuera de su bloque**.

- **Variables globales:** Son variables que no se han definido dentro de ningún bloque y por tanto **están disponibles** en cualquier parte de la **aplicación**.

Ámbito variables

```
let idGlobal=33;
let msg = 'Variable global';

function ejemplo(numero) {
 let miVarLocal = 'Soy una variable local';
 ejemplo2();
}

function ejemplo2(){
  let msg = 'Mensaje: '+idGlobal;
}
```



Clase Math

```
//Caracteristicas
Math.PI
Math.E

//Capacidades
Math.sqrt(4); //Raiz cuadrada
Math.pow(2,5); //Potencia base exponente
Math.abs(-7); //Valor absoluto
Math.random(); //Numero aleatorio decimales entre 0 y 0,9999
Math.floor(Math.random()*100); //Numero entero entre 0 y 99
Math.floor(Math.random()*100+1); //Numero entero entre 1 y 100
```

Bibliografía

- **Mozilla MDN WebDocs**
  https://developer.mozilla.org/es/
  Última visita Septiembre 2022
- **Scrimba.**
  https://www.scrimba.com
  Última visita Septiembre 2022
- **Píldoras Informáticas.**
  https://www.pildorasinformaticas.es/course/javascript-desde-0/
  Ultima visita Septiembre 2022
- **JavaScript - Tutorial.**
  https://www.w3resource.com/javascript/javascript.php
  Ultima visita Septiembre 2022
- **JavaScript - Exercises.**
  https://www.w3resource.com/javascript-exercises/
  Ultima visita Septiembre 2022
- **El Tutorial de JavaScript Moderno.**
  https://es.javascript.info/
  Ultima visita Septiembre 2022
- **Videotutorial completo JavaScript Freecodecamp.**
  https://www.youtube.com/watch?v=ivdTnPl1ND0
  Ultima visita Septiembre 2022